

Algorithms

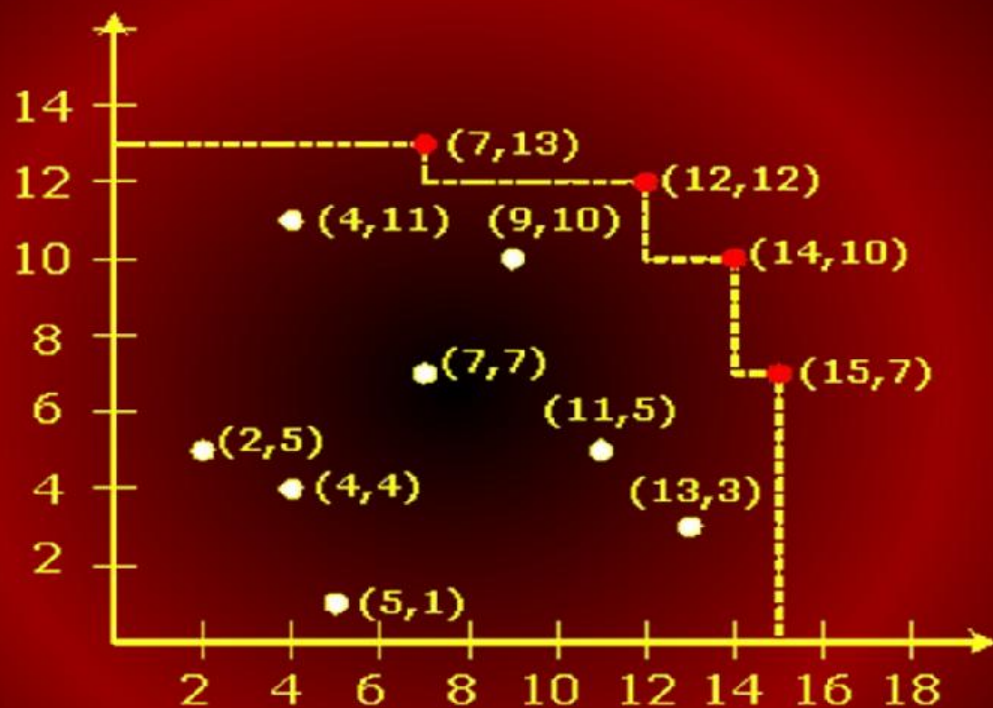
Lecture # 04

Syed Hamed Raza

Example – 2

dimension maxima

Graph



Example: 2-dimension maxima

Example: 2-dimension maxima

2-dimensional Maxima:

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space,
- output the set of maximal points of P ,
- i.e., those points p_i such that p_i is not dominated by any other point of P .

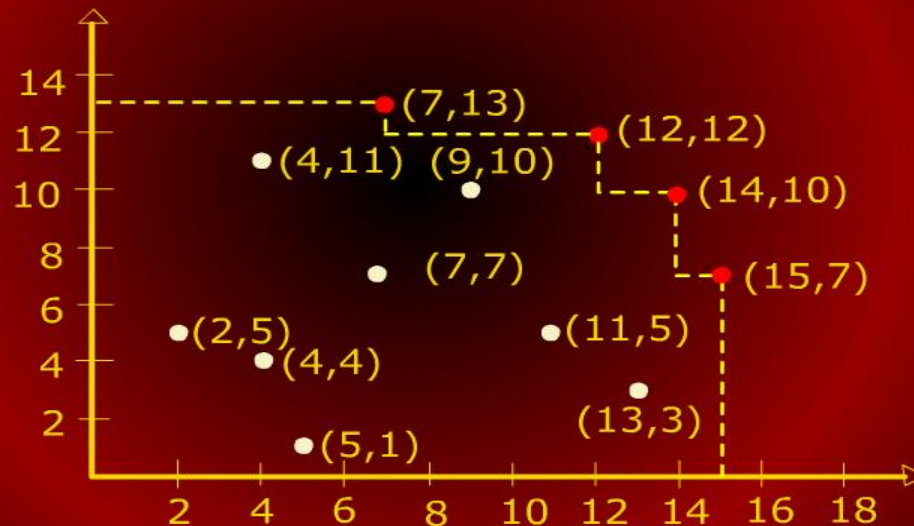
Example: 2-dimension maxima

Example: 2-dimension maxima

- Our description of the problem is at a fairly mathematical level.
- We have intentionally not discussed how the points are represented.
- We have not discussed any input or output formats.
- We have avoided programming and other software issues.

Example: 2-dimension maxima

Example: 2-dimension maxima



Example: 2-dimension maxima

Example: 2-dimension maxima

Brute-Force Algorithm:

- Let $P = \{p_1, p_2, \dots, p_n\}$ be the initial set of points.
- For each point p_i , test it against all other points p_j .
- If p_i is not dominated by any other point, then output it.

Example: 2-dimension maxima

Example: 2-dimension maxima

```
3   for  $j \leftarrow 1$  to  $n$ 
4   do
5       if  $(i \neq j)$  and  $(P[i].x \leq P[j].x)$  and
         $(P[i].y \leq P[j].y)$ 
6       then  $maximal \leftarrow false$ ; break
7   if  $(maximal = true)$ 
8       then output  $P[i]$ 
```


Example: 2-dimension maxima

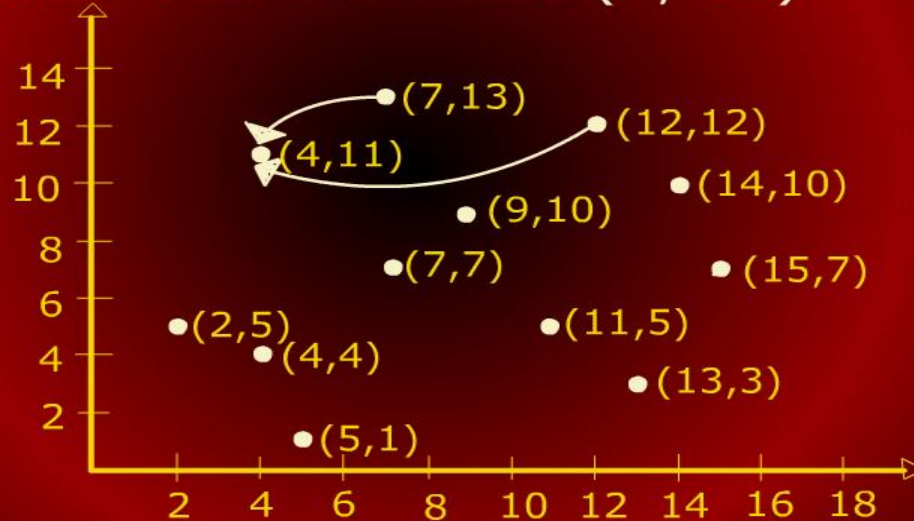
Example: 2-dimension maxima

```
MAXIMA(int n, Point P[1 . . n])  
1  for i ← 1 to n  
2  do maximal ← true  
3    for j ← 1 to n  
4    do  
5      if (i ≠ j) and (P[i].x ≤ P[j].x) and  
        (P[i].y ≤ P[j].y)  
6      then maximal ← false; break
```

Example: 2-dimension maxima

Example: 2-dimension maxima

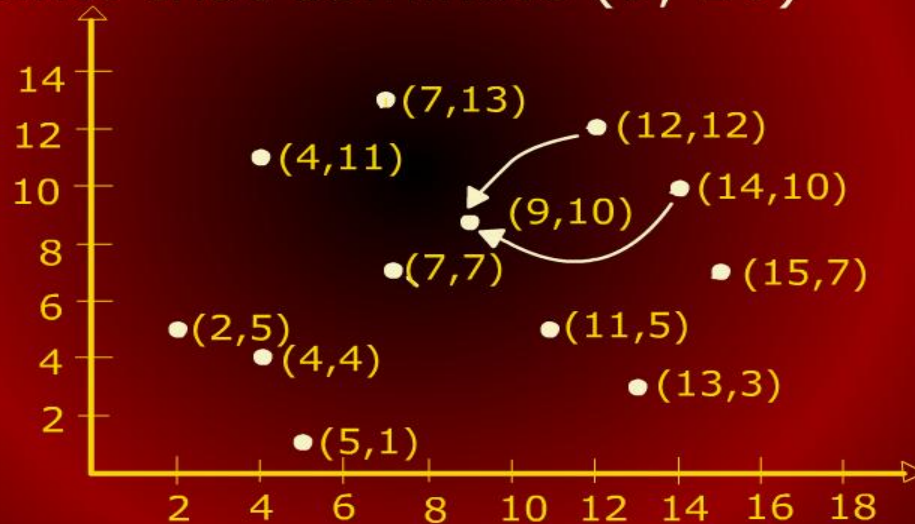
Points that dominate (4, 11)



Example: 2-dimension maxima

Example: 2-dimension maxima

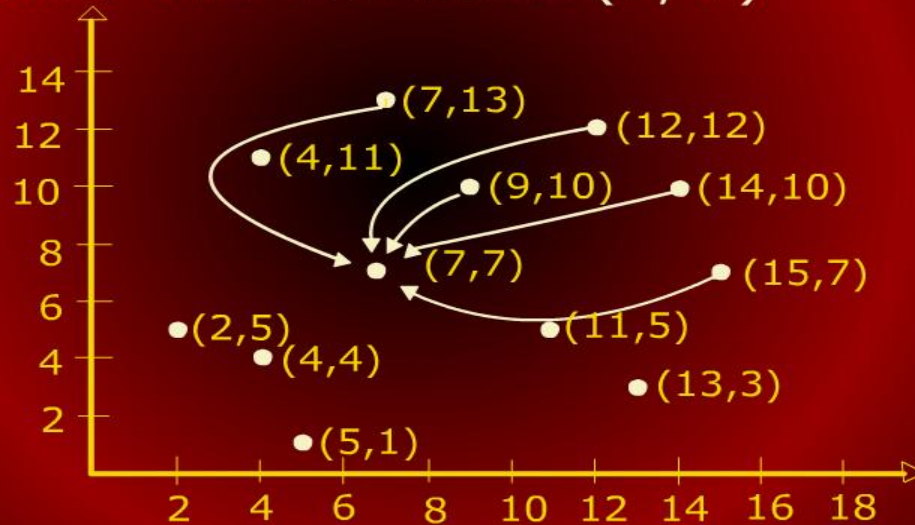
Points that dominate (9, 10)



Example: 2-dimension maxima

Example: 2-dimension maxima

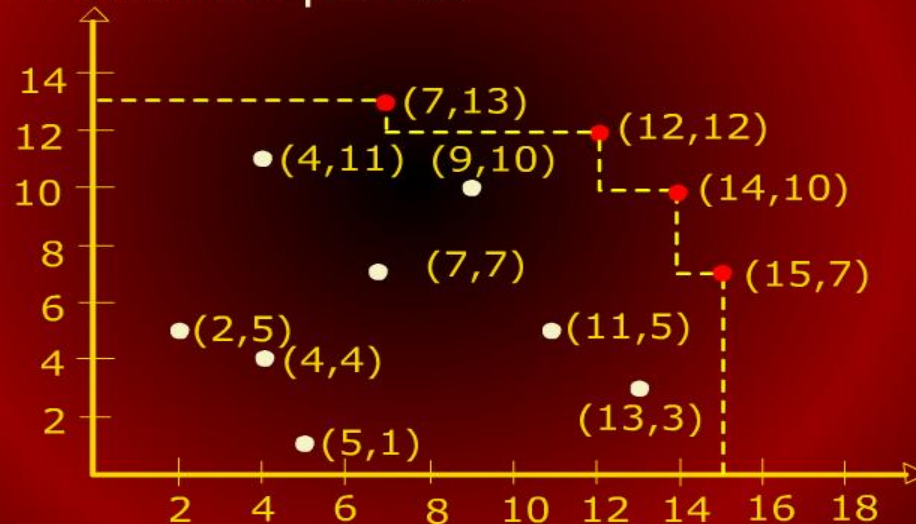
Points that dominate (7, 7)



Example: 2-dimension maxima

Example: 2-dimension maxima

The maximal points



Running Time Analysis

Running Time Analysis

Running Time Analysis

- The main purpose of our mathematical analysis will be measuring the execution time.
- We will also be concerned about the space (memory) required by the algorithm.

Running Time Analysis

Running Time Analysis

- The running time of an implementation of the algorithm would depend upon
 - the speed of the computer,
 - programming language,
 - optimization by the compiler etc.
- Although important, we will ignore these technological issues in our analysis.

Analysis of 2-d Maxima

Analysis of 2-d Maxima

To measure the running time of the brute-force 2-d maxima algorithm, we could

- count the number of steps of the pseudo code that are executed
- or, count the number of times an element of P is accessed
- or, the number of comparisons that are performed.

Running Time Analysis

Running Time Analysis

- The running time depends upon the input size, e.g. n
- Different inputs of the same size may result in different running time.
- For example, breaking out of the inner loop in the brute-force algorithm depends not only on the input size of P but also the structure of the input.

Running Time Analysis: Criteria

Running Time Analysis: Criteria

Two criteria for measuring running time:

1. Worst-case time:

- Worst-case time is the maximum running time over all (legal) inputs of size n .

Running Time Analysis: Criteria

Running Time Analysis: Criteria

Worst-case time:

- Let I denote an input instance,
- let $|I|$ denote its length, and
- let $T(I)$ denote the running time of the algorithm on input I .
- Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

Running Time Analysis: Criteria

Running Time Analysis: ***Criteria*** **2. Average-case time**

- Average-case time is the average running time over all inputs of size n .

Running Time Analysis: Criteria

Running Time Analysis: Criteria

2. Average-case time

- Let $p(I)$ denote the probability of seeing this input.
- The average-case time is the weighted sum of running times with weights being the probabilities:

$$T_{\text{avg}}(n) = \sum_{|I|=n} p(I)T(I)$$

Running Time Analysis

Running Time Analysis

- We will almost always work with worst-case time.
- Average-case time is more difficult to compute; it is difficult to specify probability distribution on inputs.
- Worst-case time will specify an upper limit on the running time.

Analysis of 2-dimension maxima

Analysis of 2-dimension maxima

Analysis of the brute-force maxima algorithm.

- Input size is n
- We will count number of times that any element of P is accessed.

Analysis of 2-dimension maxima

Example: 2-dimension maxima

```
MAXIMA(int n, Point P[1 . . . n])
1  for i ← 1 to n      n times
2  do maximal ← true
3    for j ← 1 to n    n times
4    do
5      if (i ≠ j) & (P[i].x ≤ P[j].x) &
        (P[i].y ≤ P[j].y)  4 accesses
6      then maximal ← false; break
```

Analysis of 2-dimension maxima

Analysis of 2-dimension maxima

- The outer loop runs n times
- For each i iteration, the inner loop runs n time.
- P is accessed four times in the *if* statement.
- The output statement accesses P 2 times.
- In the worst case, every point is maximal so every point is output.

Example: 2-dimension *maxima*

```
2  do maximal ← true
3    for  $j \leftarrow 1$  to  $n$   n times
4    do
5      if  $(i \neq j) \ \& \ (P[i].x \leq P[j].x) \ \&$ 
6         $(P[i].y \leq P[j].y)$   4 accesses
7      then maximal ← false; break
8    if maximal
9      then output  $P[i].x,$ 
10                $P[i].y$   2 accesses
```

